



Przedmiot:	Technologie multimedialne	IST	
		Studia: Stacjonarne	
		Semestr <u>V</u> , 2025/2026	
Temat:	Forum dyskusyjne		
Numer lab.:	17	Data wykonania:	18.04.2026
Prowadzący:	dr inż. Piotr Grad	Data oddania:	04.05.2026
Autor:	Damian Skonieczny	Indeks:	122421

Sprawozdanie - Zadanie 17 (Forum dyskusyjne)

Cel zadania

Celem było stworzenie trzypoziomowego forum dyskusyjnego w PHP i MySQL z obsługą użytkowników, moderacją treści oraz filtrowaniem niebezpiecznych linków i wulgaryzmów. Chciałem żeby to wyglądało jak prawdziwe forum, a nie jak sucha tabelka z postami.

Architektura bazy danych

Na początku zaprojektowałem bazę relacyjną, bo to w sumie serce całej aplikacji. Struktura opiera się na trzech poziomach:

- **Tematy** (`topics`) – to najwyższy poziom, coś jak kategorie. Np. "Regulamin i ogłoszenia" czy "Dyskusja ogólna".
- **Wątki** (`threads`) – zagnieżdżone w tematach. Każdy wątek ma swój tytuł i jest powiązany z konkretnym tematem kluczem obcym `topic_id`.
- **Posty** (`posts`) – odpowiedzi w wątkach. Tu ląduje treść napisana przez użytkownika. Posty mają klucz obcy `thread_id` wskazujący na wątek.

Do tego dochodzi tabela **użytkowników** (`users`) z kolumnami `role` (admin/user), `is_banned` (czy zbanowany), `profanity_count` (ile razy użył wulgaryzmu). Zastosowałem kaskadowe usuwanie (`ON DELETE CASCADE`), więc jak admin usunie temat to lecą z nim wszystkie wątki i posty, nie zostają żadne sieroty w bazie.

System rejestracji i logowania

Rejestracja i logowanie to klasyka – formularz HTML, walidacja po stronie PHP, hasła hashowane przez `password_hash()` z algorytmem `PASSWORD_DEFAULT` (bcrypt). Przy rejestracji dodałem pole "Powtórz hasło", żeby zabezpieczyć przed literówkami. Sesje trzymam w standardowy sposób przez `$_SESSION`, a w pliku `auth.php` mam funkcje

pomocnicze typu `isLoggedIn()`, `isAdmin()`, `isBanned()`, żeby nie powielać kodu w każdym widoku.

Bazowe konta w systemie:

- admin (rola: admin, hasło: admin)
- Tomek (rola: user, hasło: user)
- Jacek (rola: user, hasło: user)

Filtrowanie wulgaryzmów – klasa SecurityHelper.php

```
<?php
```

```
/**
```

```
 * z17/SecurityHelper.php
```

```
 * Klasa do obsługi cenzury wulgaryzmów oraz filtrowania niebezpiecznych linków.
```

```
 */
```

```
class SecurityHelper {
```

```
    private static $certDomainsFile = __DIR__ . '/cert_domains.txt';
```

```
    private static $certDomainsUrl =  
'https://hole.cert.pl/domains/v2/domains.txt';
```

```
    private static $cacheTime = 3600; // 1 godzina
```

```
/**
```

```
 * Zwraca true, jeśli tekst zawiera wulgaryzmy.
```

```
 */
```

```
public static function hasProfanity($text) {
```

```
    require __DIR__ . '/bad_words.php'; // ładuje $wulgaryzmy_pl
```

```
    $badWords = $wulgaryzmy_pl ?? [];
```

```
    $badWords[] = 'cholera'; // Dodajemy testowe słowo
```

```
    $textLower = mb_strtolower($text, 'UTF-8');
```

```
    // Usuwamy interpunkcję do ułatwienia sprawdzania
```

```
    $textAlphanum = preg_replace('/[^\p{L}\p{N}\s]/u', '', $textLower);
```

```
    $words = explode(' ', $textAlphanum);
```

```
    foreach ($words as $word) {
```

```
        $word = trim($word);
```

```

        if (empty($word)) continue;
        if (in_array($word, $badWords)) {
            return true;
        }
    }

    // Czasem trzeba też sprawdzić czy nie znajduje się gdzieś w środku
    (choć to może powodować false-positives np. 'komin' dla min).
    // Bezpieczniej sprawdzać jako pojedyncze słowa.
    return false;
}

/**
 * Aktualizuje zasoby domen CERT jeśli cache wygaś, i zwraca tablicę
 * domen.
 */
private static function getCertDomains() {
    if (!file_exists(self::$certDomainsFile) || (time() -
filemtime(self::$certDomainsFile)) > self::$cacheTime) {
        $content = file_get_contents(self::$certDomainsUrl);
        if ($content !== false) {
            file_put_contents(self::$certDomainsFile, $content);
        }
    }

    if (file_exists(self::$certDomainsFile)) {
        $domains = file(self::$certDomainsFile, FILE_IGNORE_NEW_LINES |
FILE_SKIP_EMPTY_LINES);
        return $domains ? $domains : [];
    }
    return [];
}

/**
 * Zastępuje niebezpieczne linki wymagany komunikatem.
 */
public static function filterBadLinks($text) {
    $domains = self::getCertDomains();
    if (empty($domains)) {
        return $text; // brak domen do odfiltrowania
    }

    // Proste wyszukiwanie linków w tekście i sprawdzanie domeny
    // łapiemy wszystkie URLe
    $pattern = '/\b(?:https?:\/\/|www\.)([^\s()<>]+(?:\([\w\d]+\)|
([^\[:punct:\]\s]|\/)))/i';
    $filteredText = preg_replace_callback($pattern, function($matches)
use ($domains) {
        $url = $matches[0];
        $host = parse_url((strpos($url, 'http') === 0 ? $url : 'http://'
. $url), PHP_URL_HOST);
        if ($host) {
            $host = preg_replace('/^www\.\/', '', $host);
            if (in_array($host, $domains)) {

```

```

        $date = date('Y-m-d H:i');
        return $date . " Usunięto niebezpieczny link";
    }
}
return $url; // Zwracamy oryginał jeśli jest bezpieczny
}, $text);
return $filteredText;
}
}
?>

```

To była chyba najtrudniejsza część zadania. Stworzyłem klasę `SecurityHelper` w osobnym pliku, żeby logika bezpieczeństwa była oddzielona od reszty kodu. Klasa ma dwie kluczowe metody:

Metoda `hasProfanity($text)`

Ładuje tablicę z pliku `bad_words.php` (baza słów wulgarnych z GitHuba, o których wspominał prowadzący w instrukcji). Tekst jest zamieniany na małe litery, usuwana jest interpunkcja, a potem każde słowo porównywane z listą. Dorzuciłem jeszcze słowo `cholera` jako dodatkowe słowo testowe – żeby łatwiej było przetestować mechanizm na prezentacji bez pisania prawdziwych brzydkich rzeczy.

Jak to działa w praktyce:

1. Użytkownik pisze post z wulgaryzmem → post **nie zostaje zapisany**.
2. W wątku pojawia się **czerwona notatka systemowa** z datą i informacją, że post usunięto za wulgaryzm. To jest pierwsze ostrzeżenie.
3. Przy **drugim** wulgarnym poście – konto użytkownika zostaje zbanowane (`is_banned = 1` w bazie) i pojawia się kolejna notatka systemowa z komunikatem o banie.
4. Po banie formularz odpowiedzi znika i wyświetla się czerwony alert.

Poniżej zrzut ekranu pokazujący komunikat systemowy po wykryciu wulgaryzmu:

Wątek: Siema. Jestem Tomek

Tomek 2026-05-04 10:05:47

Usun

Też napiszę odpowiedź tego typu

System Teraz

2026-05-04 10:05 Usunięto post użytkownika Tomek, ze względu na użyty wulgaryzm. Jest to oficjalne ostrzeżenie dla użytkownika Tomek, przy kolejnym użytym wulgaryzmie użytkownik ten zostanie zabanowany.

System Teraz

2026-05-04 10:06 Usunięto post użytkownika Tomek, ze względu na użyty wulgaryzm. Użytkownik Tomek został zabanowany z powodu używania wulgaryzmów.

Zostałeś zbanowany i nie możesz dodawać odpowiedzi.

Metoda `filterBadLinks($text)`

Ta metoda obsługuje filtrowanie niebezpiecznych domen z listy CERT Polska. Na początku miałem problem – za każdym razem gdy ktoś dodawał post, skrypt pobierał plik `domains.txt` z serwera CERT, co strasznie zamulało stronę. Rozwiązałem to **cache'owaniem**: lista domen jest pobierana z internetu maksymalnie raz na godzinę i zapisywana lokalnie do pliku `cert_domains.txt`. Przy każdym kolejnym poście skrypt czyta już z dysku.

Mechanizm wyciąga z tekstu wszystkie URLe za pomocą wyrażenia regularnego, sprawdza domenę pod kątem listy CERT i jeśli trafi na niebezpieczną – zamienia link na komunikat w stylu: `2026-04-12 15:30 Usunięto niebezpieczny link.`

Dodawanie postów przez AJAX (Fetch API)

Przebudowałem formularz dodawania postów tak, żeby korzystał z AJAX-a. Dzięki temu strona nie przeładowuje się po wysłaniu odpowiedzi – post pojawia się dynamicznie na dole wątku. Użyłem do tego natywnego `Fetch API` ze współczesnego JavaScriptu oraz obiektu `FormData`, który obsługuje też wysyłanie plików (zdjęć, filmów itp.).

Cały flow wygląda tak:

1. Użytkownik wpisuje treść i kliknie "Wyślij odpowiedź".
2. JavaScript wysyła żądanie POST do `thread.php` z parametrem `ajax=1`.
3. Serwer przetwarza post (filtrowanie, zapis do bazy) i zwraca odpowiedź JSON.
4. JS odbiera JSON i dynamicznie wstawia nowy post do DOM-u strony.

Ważne było, żeby logika AJAX-owa wykonywała się **przed** dołączeniem `header.php` (który generuje HTML), bo inaczej odpowiedź serwera zawierała HTML przed JSON-em i przeglądarka nie potrafiła tego sparsować.

Obsługa załączników multimedialnych

Posty mogą mieć załączone pliki multimedialne (zdjęcia, filmy). Plik jest uploadowany do katalogu `/uploads/` z unikatową nazwą (prefix `time()`), a ścieżka zapisywana w kolumnie `media_path` tabeli `posts`. W widoku wątku załącznik wyświetla się pod treścią posta jako responsywny obraz.

Panel Administratora

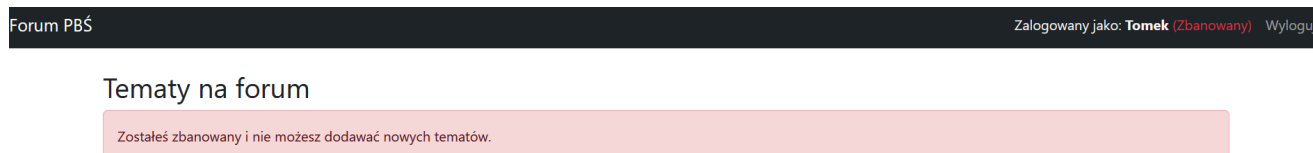
Admin ma dostęp do specjalnej zakładki `admin.php` z panelem zarządzania. Panel zawiera:

- **Tabelę użytkowników** – z informacjami o roli, statusie (aktywny/zbanowany), liczbie wykrytych wulgaryzmów. Przyciski: Zbanuj/Odbanuj, Usuń użytkownika, Usuń posty użytkownika.
- **Listę tematów** – z możliwością usunięcia (kaskadowo usuwa wątki i posty).

- **Listę wątków** – z możliwością usunięcia (kaskadowo usuwa posty).

Wszystkie akcje destrukcyjne mają potwierdzenie (`confirm()`), żeby nie usunąć czegoś przez przypadek.

Poniżej zrzut ekranu widoku zbanowanego użytkownika na forum:



Frontend i responsywność

Do stylowania użyłem Bootstrapa 5 (podpięty przez CDN). Posty mają kolorowe obramowania z lewej strony – niebieskie dla zwykłych postów, czerwone dla postów systemowych. Nawigacja jest responsywna (hamburger menu na mobilkach). Breadcrumbs pomagają w nawigacji: Forum → Temat → Wątek.

Konfiguracja środowiska

Połączenie z bazą danych jest skonfigurowane przez plik `.env`, zgodnie z szablonem z `.env.example`. Dzięki temu mogę płynnie przełączać się między środowiskiem lokalnym a hostingiem, bez ręcznej edycji kodu.

```
DB_HOST=127.0.0.1
DB_NAME=damskopb_z17
DB_USER=root
DB_PASS=
```

Podsumowanie

Na początku myślałem, że forum to nic skomplikowanego – w końcu formularze rejestracji robiliśmy już kilka razy. Ale jak doszły wulgaryzmy, lista CERT, AJAX i panel admina, to zrobiło się naprawdę dużo roboty. Najbardziej namęczyłem się z tym cache'owaniem listy domen i z debugowaniem AJAX-a (bo `header.php` generował HTML przed JSON-em). Całość wygląda przyzwoicie i Bootstrap ładnie to układa wizualnie. Podrzucam na repozytorium!